

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERIA



Centro Universitario De Ciencias Exactas e Ingenierías

Análisis de Algoritmos

Fecha: 09/09/2025

Tema: ACT. Visualizador de Métodos de Ordenamiento

Alumno(s): Rosas Arreola Ricardo Alejandro.

Owen Alexis Leal Martínez

Sección: D01

NRC / Clave: 204835 / IL355

Código(s): 222967177 / 219554686

Carrera: Ingeniería en Computación.

División de Tecnologías para la Integración Ciber-Humana

Maestro/a: JORGE ERNESTO LOPEZ ARCE DELGADO.

Tabla de contenido

Introducción	3
Objetivo(s).....	4
Desarrollo	5
Conclusiones	7
Referencias	8

Introducción

Los algoritmos de ordenamiento son herramientas fundamentales para todos los programadores y es de suma importancia conocer su comportamiento. Debido a ello en la siguiente actividad conoceremos 4 diferentes algoritmos:

Bubble sort, Quick Sort, Merge Sort y Selection Sort.

Son algoritmos de ordenamiento que podremos visualizar gráficamente en un programa de Python.

Objetivo(s)

1. Implementar en Python los algoritmos Bubble Sort, Merge Sort y Selection Sort.
2. Permitir la selección del algoritmo a utilizar para ordenar un conjunto de datos.
3. Visualizar gráficamente el proceso de ordenamiento para comprender mejor el funcionamiento interno de cada algoritmo.
4. Comparar las diferencias en tiempo de ejecución y eficiencia de los algoritmos.

Desarrollo

Para el programa, se diseñó una gráfica el cual muestra una lista con números aleatorios representados con barras, el cual se ordena por medio de los diferentes métodos de ordenamiento, se generan a partir de una N cantidad (el cual lo genera el usuario) y mediante una serie de botones puede manejar a su disposición lo que desea hacer con la lista, ya sea Generar, Mezclar, Ordenar y Limpiar, Puede seleccionar a su preferencia el algoritmo a utilizar y especificar la velocidad en la que se ordena.

De forma mas clara y breve:

Dropdown: Desprende con un click los diferentes algoritmos de ordenamiento

Boton de generar: Genera datos aleatorios

Botón de ordenar: Ordena los datos con el algoritmo seleccionado

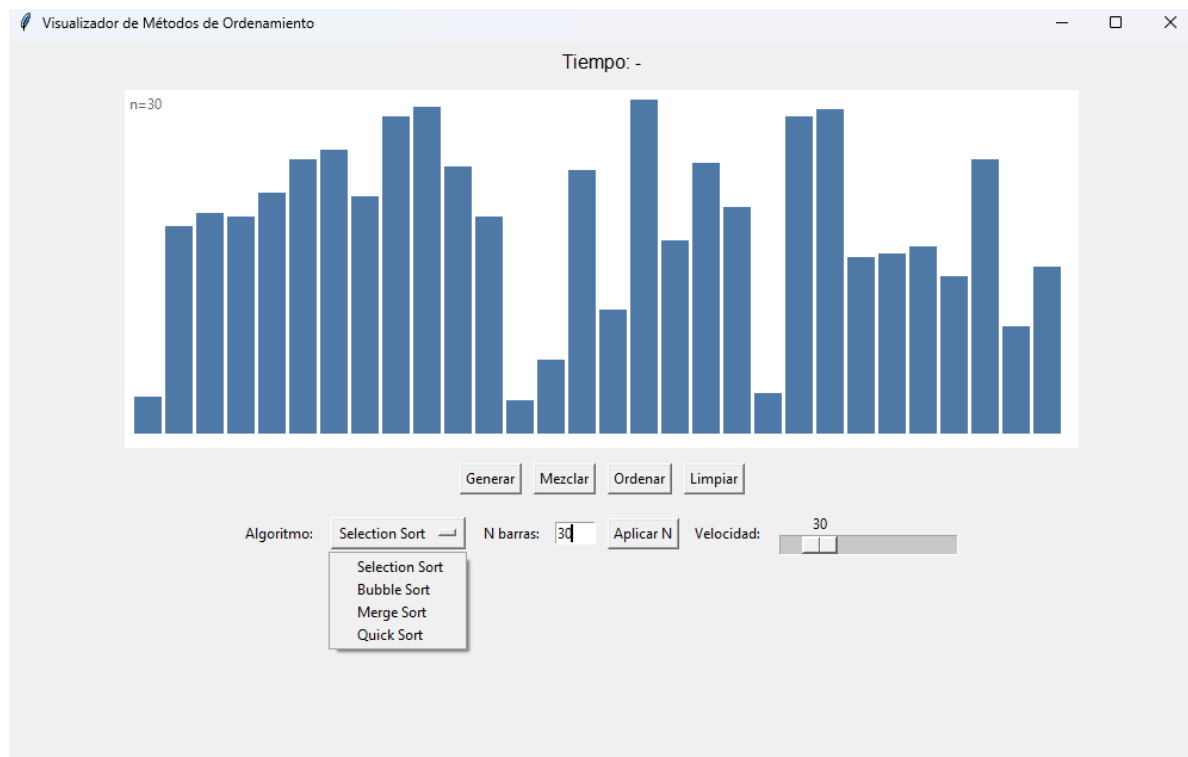
Boton de mezclar: Barajea los datos actuales datos al azar

Botón de limpiar:

Detiene el programa y limpia los colores gráficos

Scale: permite seleccionar la velocidad con la que el algoritmo ordena los datos

Visualización general del programa:



Ejemplo de funcionamiento con Merge Sort:

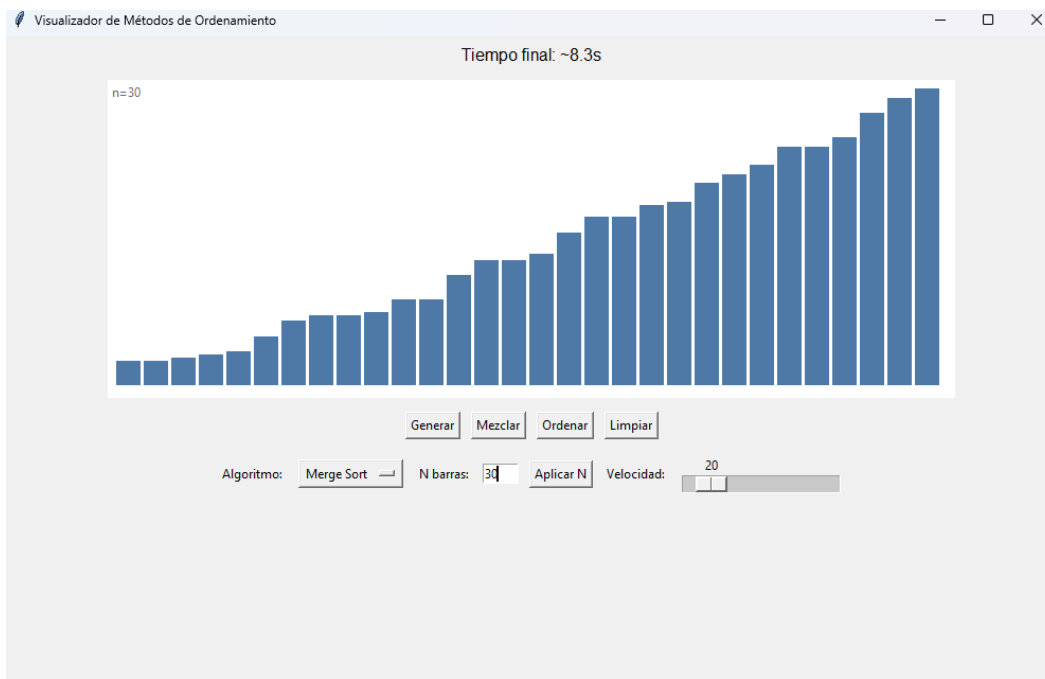


Tabla de N cantidades con sus tiempos aproximados de ordenamiento (dependiendo del algoritmo utilizado)

N	Selection sort	Bubble Sort	Quick Sort	Merge Sort
5	0.4s	0.5s	0.3s	0.6s
10	1.3s	1s	1.3s	1.9s
15	3.4s	4.6s	1.9s	3.2s
20	5.9s	8s	3.8s	4.8s

Comentarios:

El Selection Sort: Empieza a decaer en tiempo cuando son conjuntos o cantidades más grandes, y cada vez que sean mas elementos, el tiempo de ejecución se vuelve cuadrática.

Bubble Sort: Es de los mas lentos a grandes conjuntos o cantidades, realmente sirve más como base para aprender sobre los métodos de ordenamiento, no es muy recomendable.

Quick Sort: Es de los mas eficientes para grandes conjuntos o cantidades, es de las más recomendadas para utilizar.

Merge sort: También es una alternativa viable para grandes conjuntos y cantidades.

Conclusiones

Después de llevar a cabo esta actividad, puedo recabar que los distintos algoritmos de ordenamiento sirven más o menos dependiendo de la lista a ordenar, por ejemplo bubble Sort la mayoría de veces tarda más que Merge sort.

En conclusión, este programa nos enseñó que algoritmos son más eficientes para cierta cantidad de datos.

Referencias

González Duque, R. (2014). Python para todos.

Moore, A. D. (2018). Python GUI Programming with Tkinter: Develop responsive and powerful GUI applications with Tkinter. Packt Publishing Ltd.

Van Rossum, G., & Drake, F. L. (1995, April). Python library reference.